SISTEMA DE VENDAS · VENDAS-API

Login na API com Spring Boot

Recebendo o login, conferindo a senha e emitindo o token, passo a passo: implementar, testar e entender.



VISÃO GERAL

O que vamos construir

REQUISIÇÃO

```
POST /api/auth/login
{ "usuario": "admin", "senha": "admin123" }
```

RESPOSTA 200

```
{ "token": "eyJhbGciOi...",
  "usuario": { "nome": "Administrador" } }
```

RESPOSTA 401

```
{ "message": "Usuário ou senha inválidos." }
```

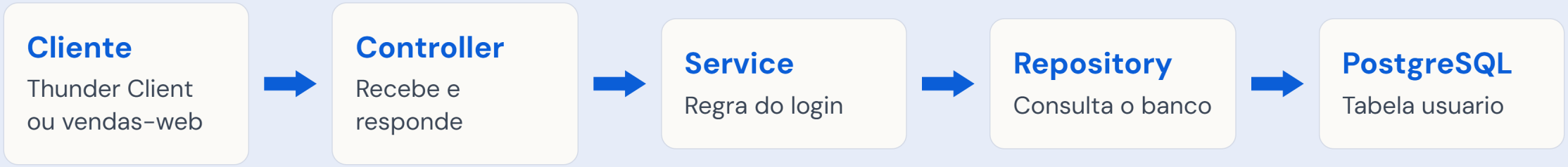
Ao final você terá

- Um endpoint POST de login em Spring
- Entrada validada com Bean Validation
- Usuários no PostgreSQL via JPA
- Senha guardada como hash BCrypt
- Token JWT assinado, com validade

Método: implementar, testar, entender

ANTES DO CÓDIGO

As camadas da API



Cada camada tem uma única responsabilidade. Vamos construir na ordem do caminho da requisição: da porta de entrada até o banco.

ROTEIRO

Seis etapas, cada uma com teste

Etapa	O que construímos	Classes	Como testamos
1	Endpoint com resposta fixa	AuthController, DTOs	POST devolve 200 fixo
2	Validação da entrada	LoginRequest	Corpo vazio devolve 400
3	Usuário no banco	Usuario, Repository, Service	admin: 200, outro: 401
4	Senha com BCrypt	PasswordConfig	Senha errada devolve 401
5	Token JWT	TokenService	Abrir o token no jwt.io
6	Integração com a tela	vendas-web	Entrar pela tela de login

Em cada etapa: escrevemos pouco código, testamos no Thunder Client e só então explicamos o que aconteceu.

ETAPA 1 DE 6

Endpoint visível

Primeiro a API responde. Ainda com dados fixos, sem banco e sem senha.

Pronto quando: o Thunder Client recebe 200 com token e nome

CONCEITOS DA ETAPA 1

Uma API REST em Spring

@RestController

Marca a classe como controlador REST: o retorno dos métodos vira JSON na resposta.

@PostMapping

Liga um método a uma URL e ao verbo POST. O prefixo da classe vem de @RequestMapping.

@RequestBody

Converte o JSON do corpo da requisição em um objeto Java.

record e DTO

Classe imutável e enxuta, ideal para transportar dados. DTO significa Data Transfer Object.

ETAPA 1 · PACOTE DTO

Os DTOs do contrato

NOVO ARQUIVO · dto/LoginRequest.java

```
public record LoginRequest(String usuario, String senha) {}
```

NOVO ARQUIVO · dto/LoginResponse.java

```
public record LoginResponse(String token, UsuarioResumo usuario) {
  public record UsuarioResumo(String nome) {}
}
```

Os nomes dos campos são os nomes do JSON.

record gera construtor e acessores sozinho.

UsuarioResumo é o objeto aninhado usuario.nome.

ETAPA 1 · PACOTE CONTROLLER

O AuthController com resposta fixa

NOVO ARQUIVO · controller/AuthController.java

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

  @PostMapping("/login")
  public LoginResponse login(@RequestBody LoginRequest request) {
    var usuario = new LoginResponse.UsuarioResumo("Administrador");
    return new LoginResponse("token-fixo", usuario);
  }
}
```

@RequestMapping define o prefixo /api/auth.

@PostMapping responde só a POST em /login.

O retorno vira JSON automaticamente.

TESTE DA ETAPA 1

Primeira chamada no Thunder Client

Faça

1. Rode a VendasApiApplication
2. Crie um POST para localhost:8080/api/auth/login
3. Em Body, escolha JSON e envie {"usuario":"admin","senha":"x"}

Esperado

- Status 200 OK
- Resposta com o token-fixo
- usuario.nome igual a Administrador

A API já responde no formato que o vendas-web espera, mas aceita qualquer corpo, até campos vazios. A etapa 2 corrige isso.

ETAPA 2 DE 6

Validação

A API não confia no que chega: dados inválidos são recusados antes de qualquer lógica.

Pronto quando: campos vazios recebem 400 Bad Request

CONCEITOS DA ETAPA 2

Nunca confie na entrada

Bean Validation

Anotações como @NotBlank que descrevem regras do dado. O starter validation já está no pom.

@Valid

Pede ao Spring que aplique as regras ao objeto recebido, antes de o método ser executado.

@NotBlank

O texto não pode ser nulo, vazio nem conter só espaços.

400 Bad Request

Status HTTP que diz que o cliente enviou algo errado. A requisição nem chega ao método.

ETAPA 2 · LOGINREQUEST E AUTHCONTROLLER

Anotar o DTO e ativar o @Valid

DTO/LOGINREQUEST.JAVA · SUBSTITUA O RECORD

```
public record LoginRequest(
    @NotBlank String usuario,
    @NotBlank String senha) {}
```

CONTROLLER/AUTHCONTROLLER.JAVA · ACRESCENTE @VALID

```
public LoginResponse login(@Valid @RequestBody LoginRequest request) {
```

Os imports vêm de jakarta.validation (NotBlank e Valid). O pacote é jakarta, não javax.

TESTE DA ETAPA 2

Corpo válido e inválido

Faça

1. Envie {"usuario":"","senha":""}
2. Envie só {"usuario":"admin"}
3. Envie usuário e senha preenchidos

Esperado

- Os dois primeiros devolvem 400 Bad Request
- O terceiro devolve 200 como antes
- O corpo do 400 é o padrão do Spring

A entrada agora é confiável. Falta saber quem é o usuário: etapa 3, com o banco de dados.

ETAPA 3 DE 6

Usuário no banco

Agora o login consulta a tabela usuario no PostgreSQL, usando JPA.

Pronto quando: admin retorna 200 e qualquer outro usuário retorna 401

CONCEITOS DA ETAPA 3

Persistência com JPA

Entity

Classe Java mapeada para uma tabela. Cada objeto representa uma linha.

Repository

Interface que o Spring implementa sozinho: salvar, buscar e remover sem escrever SQL.

Query method

findByLogin vira SELECT ... WHERE login = ? só pelo nome do método.

Optional

Caixa que pode ou não ter valor. Evita NullPointerException quando o usuário não existe.

ETAPA 3 · BANCO DE DADOS

Criar a tabela e o usuário admin

SQL · EXECUTE NO POSTGRESQL

```
CREATE TABLE usuario (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  login VARCHAR(50) NOT NULL UNIQUE,
  senha_hash VARCHAR(100) NOT NULL,
  nome VARCHAR(100) NOT NULL
  );

INSERT INTO usuario (login, senha_hash, nome) VALUES
  ('admin', '$2a$10$PiwnsPbXhQQCzd4AYnhDlO0uZT2NYica1oThTV1iyXfbejCLod51y', 'Administrador');
```

A coluna se chama senha_hash porque a senha nunca é guardada em texto puro. Esse hash corresponde à senha admin123 (veremos na etapa 4).

ETAPA 3 · PACOTE MODEL

A entidade Usuario

NOVO ARQUIVO · model/Usuario.java

```
@Entity
@Table(name = "usuario")
public class Usuario {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;
  private String login;
  @Column(name = "senha_hash")
  private String senhaHash;
  private String nome;

  // getters: getLogin(), getSenhaHash(), getNome()
}
```

@Entity marca a classe como tabela.

@Id é a chave primária.

@Column liga senhaHash a senha_hash.

ETAPA 3 · PACOTE REPOSITORY

O UsuarioRepository

NOVO ARQUIVO · repository/UsuarioRepository.java

```
public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

  Optional<Usuario> findByLogin(String login);
}
```

JpaRepository<Usuario, Long> já traz save, findById, findAll e delete. O findByLogin é gerado pelo Spring a partir do nome do método.

ETAPA 3 · APPLICATION.PROPERTIES

Conferir a configuração do banco

SRC/MAIN/RESOURCES/APPLICATION.PROPERTIES

```
spring.datasource.url=jdbc:postgresql://localhost:5432/vendas
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=validate
server.error.include-message=always
```

validate: o Hibernate só confere se a tabela bate com a entidade. include-message: devolve a mensagem do erro 401 no JSON.

ETAPA 3 · PACOTE SERVICE

O AuthService: dependência por construtor

NOVO ARQUIVO · service/AuthService.java (ESQUELETO)

```
@Service
public class AuthService {

  private final UsuarioRepository repository;

  public AuthService(UsuarioRepository repository) {
    this.repository = repository;
  }
}
```

@Service registra a classe no Spring.

Injeção o Spring entrega o repository ao construtor.

final a dependência não muda depois.

ETAPA 3 · AUTHSERVICE

O método login

service/AuthService.java · ACRESCENTE O MÉTODO

```
public LoginResponse login(LoginRequest request) {

  var usuario = repository.findByLogin(request.usuario())
    .orElseThrow(() → new ResponseStatusException(
      HttpStatus.UNAUTHORIZED, "Usuário ou senha inválidos."));

  var resumo = new LoginResponse.UsuarioResumo(usuario.getNome());
  return new LoginResponse("token-fixo", resumo);
}
```

orElseThrow: se o Optional estiver vazio, lança a exceção. ResponseStatusException vira a resposta HTTP 401 Unauthorized.

ETAPA 3 · AUTHCONTROLLER

O controller delega ao service

controller/AuthController.java · SUBSTITUA O CORPO

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

  private final AuthService authService;

  public AuthController(AuthService authService) {
    this.authService = authService;
  }

  @PostMapping("/login")
  public LoginResponse login(@Valid @RequestBody LoginRequest request) {
    return authService.login(request);
  }
}
```

TESTE DA ETAPA 3

Usuário existente e inexistente

Faça

1. Reinicie a aplicação
2. Envie usuario admin com qualquer senha
3. Envie usuario fulano com qualquer senha

Esperado

- admin: 200 com nome Administrador
- fulano: 401 Unauthorized
- Mensagem: Usuário ou senha inválidos.

Pegadinha proposital: admin com senha errada ainda entra. Corrigimos agora, com BCrypt.

ETAPA 4 DE 6

Senha com BCrypt

Comparar a senha digitada com o hash guardado no banco.

Pronto quando: senha errada retorna 401 e admin123 retorna 200

CONCEITOS DA ETAPA 4

Hash de senha

Hash

Função de mão única: da senha sai um texto fixo, mas do texto não se volta à senha.

BCrypt

Algoritmo feito para senhas: lento de propósito, o que dificulta ataques de força bruta.

Salt

Valor aleatório embutido no hash. Duas senhas iguais geram hashes diferentes.

matches

Refaz o hash da senha digitada usando o salt e compara. Nunca se decifra o hash.

ETAPA 4 · POM.XML

Adicionar a dependência

POM.XML · DENTRO DE <DEPENDENCIES>

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-crypto</artifactId>
</dependency>
```

Só o módulo de criptografia, sem a versão (o Spring Boot já a gerencia). Não é o Spring Security completo: nenhum filtro é ativado e a API continua aberta.

ETAPA 4 · PACOTE CONFIG

Disponibilizar o PasswordEncoder

NOVO ARQUIVO · config/PasswordConfig.java

```
@Configuration
public class PasswordConfig {

  @Bean
  public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
  }
}
```

@Configuration classe que declara beans.

@Bean o objeto retornado fica disponível para injeção.

PasswordEncoder a interface; BCrypt é a implementação.

ETAPA 4 · AUTHSERVICE

Injetar o PasswordEncoder

service/AuthService.java · SUBSTITUA CAMPOS E CONSTRUTOR

```
private final UsuarioRepository repository;
private final PasswordEncoder passwordEncoder;

public AuthService(UsuarioRepository repository,
PasswordEncoder passwordEncoder) {
  this.repository = repository;
  this.passwordEncoder = passwordEncoder;
}
```

ETAPA 4 · AUTHSERVICE

Conferir a senha no login

service/AuthService.java · AJUSTE O MÉTODO login

```
var usuario = repository.findByLogin(request.usuario())
  .filter(u → passwordEncoder.matches(
      request.senha(), u.getSenhaHash()))
  .orElseThrow(() → new ResponseStatusException(
      HttpStatus.UNAUTHORIZED, "Usuário ou senha inválidos."));
```

filter mantém o usuário só se a senha confere. Se não conferir, o Optional fica vazio e o mesmo orElseThrow responde 401.

TESTE DA ETAPA 4

Senha certa e errada

Faça

1. admin com senha errada
2. admin com senha admin123
3. fulano com qualquer senha

Esperado

- Senha errada: 401
- admin123: 200 com Administrador
- fulano: 401

A autenticação agora é real. Falta devolver um token de verdade em vez do token-fixo.

ETAPA 5 DE 6

Token JWT

Em vez do token fixo, a API devolve um token assinado, com validade.

Pronto quando: o token aparece decodificado no jwt.io, com o nome do usuário

CONCEITOS DA ETAPA 5

O que é um JWT

Token

Texto que a API entrega após o login. Prova que o usuário já se autenticou.

JWT

JSON Web Token: formato padrão de token com três partes separadas por ponto.

Claims

Informações dentro do token: quem é o usuário (sub), o nome e quando expira (exp).

Assinatura

Calculada com um segredo que só o servidor conhece. Se alguém alterar o conteúdo, a assinatura deixa de bater.

ETAPA 5 · ANATOMIA

Um JWT tem três partes

EXEMPLO ABREVIADO

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJhZG1pbiJ9 . Qx7r...k2
```

Header

Algoritmo e tipo. Aqui, HS256.

Payload

Os claims: {"sub":"admin","nome":"Administrador","exp":...}

Signature

Hash do header e do payload com o segredo.

Header e payload são JSON codificado em Base64. Qualquer pessoa consegue ler: por isso nada sigiloso entra no token.

ETAPA 5 · POM.XML E PROPERTIES

Biblioteca e configurações

POM.XML · NOVA DEPENDÊNCIA

```
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.5.0</version>
</dependency>
```

APPLICATION.PROPERTIES

```
app.jwt.secret=${JWT_SECRET:troque-esta-chave-em-producao-minimo-32-caracteres}
app.jwt.expiracao-minutos=60
```

O segredo vem da variável de ambiente JWT_SECRET; o valor depois dos dois pontos é só um padrão para desenvolvimento.

ETAPA 5 · PACOTE SERVICE

TokenService: configuração

NOVO ARQUIVO · service/TokenService.java (ESQUELETO)

```
@Service
public class TokenService {

  private final Algorithm algoritmo;
  private final long expiracaoMinutos;

  public TokenService(
      @Value("${app.jwt.secret}") String segredo,
      @Value("${app.jwt.expiracao-minutos}") long expiracaoMinutos) {
    this.algoritmo = Algorithm.HMAC256(segredo);
    this.expiracaoMinutos = expiracaoMinutos;
  }
}
```

@Value lê uma propriedade do application.properties. HMAC256 é o algoritmo da assinatura, feito a partir do segredo.

ETAPA 5 · TOKENSERVICE

Gerar o token

service/TokenService.java · ACRESCENTE O MÉTODO

```
public String gerar(Usuario usuario) {
  return JWT.create()
    .withSubject(usuario.getLogin())
    .withClaim("nome", usuario.getNome())
    .withExpiresAt(Instant.now()
      .plus(expiracaoMinutos, ChronoUnit.MINUTES))
    .sign(algoritmo);
}
```

withSubject quem é o dono do token.

withClaim dado extra: o nome.

withExpiresAt validade de 60 minutos.

ETAPA 5 · AUTHSERVICE

Trocar o token fixo pelo real

service/AuthService.java · INJETE O TokenService E USE

```
// novo campo e parâmetro no construtor:
private final TokenService tokenService;

// no final do método login:
var token = tokenService.gerar(usuario);
return new LoginResponse(token, resumo);
```

O AuthService decide se o login vale; o TokenService só sabe fabricar tokens. Cada classe, uma responsabilidade.

TESTE DA ETAPA 5

Abrir o token no jwt.io

Faça

1. Faça login com admin e admin123
2. Copie o valor do campo token
3. Cole em jwt.io, no campo Encoded

Esperado

- Header com alg HS256
- Payload com sub admin e nome Administrador
- exp cerca de 60 minutos à frente

A API entrega o token. Resta a tela real consumi-lo, na etapa 6.

ETAPA 6 DE 6

Integração com a tela

O vendas-web passa a chamar a API de verdade, no lugar do login simulado.

Pronto quando: a tela de login entra com admin e admin123

CONCEITOS DA ETAPA 6

Duas aplicações, um contrato

Dois servidores

A API em localhost:8080 e o Vite em localhost:5173 precisam estar rodando juntos.

Proxy do Vite

Chamadas para /api na tela são repassadas ao 8080. Para o navegador, a origem é uma só.

CORS

Bloqueio do navegador entre origens diferentes. O proxy evita configurar isso agora.

Contrato

O que a tela envia e espera receber: usuario e senha na ida, token e usuario.nome na volta.

TESTE DA ETAPA 6

Login pela tela

Faça

1. Suba a API e o vendas-web
2. Abra localhost:5173/login
3. Entre com admin e admin123, depois com senha errada

Esperado

- Credenciais corretas: vai para a Home
- Senha errada: mensagem Usuário ou senha inválidos.
- DevTools, aba Network: POST /api/auth/login 200 e 401

O ciclo completo funciona: tela, API, banco, hash e token.

RESUMO

O caminho do login

1

Tela envia

POST /api/auth/login

2

Controller valida

@Valid, 400

3

Service busca

findByLogin

4

BCrypt confere

matches, 401

5

TokenService gera

JWT assinado

6

Tela guarda

token na sessão

GLOSSÁRIO

Termos desta aula

Endpoint
URL que a API expõe para um verbo HTTP.

DTO
Objeto que transporta dados na entrada e saída.

record
Classe imutável e concisa do Java.

Bean Validation
Anotações que validam dados.

Entity
Classe mapeada a uma tabela.

Repository
Interface de acesso ao banco.

Injeção de dependência
O Spring entrega os objetos que a classe precisa.

Bean
Objeto gerenciado pelo Spring.

Hash
Resumo de mão única de um texto.

Salt
Aleatoriedade que torna cada hash único.

JWT
Token com claims e assinatura.

Claim
Informação dentro do token.

PRÓXIMOS PASSOS

Desafios e próxima aula

Proteger rotas

Filtro que valida o token nas demais requisições. É a próxima aula.

Cadastro de usuário

Endpoint que grava um novo usuário com a senha em BCrypt.

Segredos e HTTPS

Segredo por variável de ambiente e tráfego sempre cifrado em produção.

Limite de tentativas

Bloquear tentativas repetidas de login.